

动态数组升级 2

C++

```
1  #include<iostream>
2  using namespace std;
3  template <class T>
4  class DynamicVector
5  {
6      T* array; // pointer to the items 指向分配空间的指针
7      unsigned mallocSize; // number of allocated spaces 分配空间的大小
8      unsigned numofItems; // number of items 向量内已经存储的元素数量
9      int virtualZero; // virtual zero 数组起始下标, C语言中通常数组下标是从0开始, 这个数据属性可以让数组的下标从-10或2等
        等整数开始, 让数组更加灵活。
10
11     public:
12
13         //构造函数
14         DynamicVector (int );
15
16         //copy构造函数
17         DynamicVector(const DynamicVector&);
18
19         //析构函数
20         ~DynamicVector();
21
22         //添加元素或改变空间大小
23         inline void push_back (const T&newItem);
24
25         //push_back重载
26         void push_back (const DynamicVector&other);
27
28         //operator[]重载
29         T& operator[] (int Vindex);
30
31         //成员函数 - 返回length
32         unsigned length() const;
33
34         //成员函数 - 返回capacity
35         unsigned capacity() const;
36
37         //成员函数 - 返回下标
38         int firstIndex() const;
39
40         //operator=重载
41         DynamicVector& operator=(const DynamicVector&a);
42
43         //逻辑==重载
44         bool operator == (const DynamicVector<T>& dv) const;
45     };
46
47     //构造函数
48     template <class T>
49     DynamicVector<T>::DynamicVector(int Vindex)
50     {
51         array = NULL;
52         numofItems = 0;
53         mallocSize = 0;
54         virtualZero = Vindex;
55     }
56
57     //copy构造函数
58     template <class T>
```

```

59 DynamicVector<T>::DynamicVector(const DynamicVector&a)
60 {
61     //初始化当前向量的属性
62     mallocSize=a.mallocSize;
63     numofItems=a.numofItems;
64     virtualZero=a.virtualZero;
65     //申请空间
66     array=new T[mallocSize];
67     //复制其他向量的元素到当前向量
68     for (unsigned i=0;i<numofItems;i++)
69     {
70         array[i]=a.array[i];
71     }
72 };
73
74 //析构函数
75 template <class T>
76 DynamicVector<T>::~~DynamicVector()
77 {
78     delete[] array;
79 };
80
81 //添加元素或改变空间大小
82 template <class T>
83 void DynamicVector<T>::push_back (const T&newItem)
84 {
85     //已经储存的元素数量小于所分配的空间大小
86     if(numofItems<mallocSize)
87     {
88         //直接在已知元素的最后添加newItem元素
89         array[numofItems]=newItem;
90         numofItems++;
91     }
92     //已经储存的元素数量等于所分配的空间大小
93     else if(numofItems==mallocSize)
94     {
95         //分配新空间
96         T* newarray=new T[2*mallocSize+1];
97         //复制原来的数据
98         for(unsigned int i=0;i<mallocSize;i++)
99         {
100             newarray[i]=array[i];
101         }
102         //在已知元素的最后添加newItem元素
103         newarray[mallocSize]=newItem;
104         //释放内存
105         delete[] array;
106         //更新 - 不更新的话只输出
107         array=newarray;
108         mallocSize=2*mallocSize+1;
109         numofItems++;
110     }
111 };
112
113 //push_back重载
114 template <class T>
115 void DynamicVector<T>::push_back (const DynamicVector&other)
116 {
117     //通过循环将其他向量中的元素逐个添加到当前向量中
118     for (unsigned i=0;i<other.numofItems;i++)
119     {
120         push_back(other.array[i]);
121     }
122 };
123
124 //operator[]重载
125 template <class T>
126 T& DynamicVector<T>::operator[] (int Vindex)
127 {
128     int _entry = Vindex - virtualZero;

```

```

129     if (_entry < 0 || _entry ≥ numofItems)
130     {
131         cout<<endl<<"Out Of Range";
132         exit(1);
133     }
134     return array[_entry];
135 }
136
137 //成员函数 - 返回length
138 template <class T>
139 unsigned DynamicVector<T>::length() const
140 {
141     return numofItems;
142 }
143
144 //成员函数 - 返回capacity
145 template <class T>
146 unsigned DynamicVector<T>::capacity() const
147 {
148     return this->mallocSize;
149 }
150
151 //成员函数 - 返回下标
152 template <class T>
153 int DynamicVector<T>::firstIndex() const
154 {
155     return this->virtualZero;
156 }
157
158 //operator=重载
159 template <class T>
160 DynamicVector<T>& DynamicVector<T>::operator=(const DynamicVector&a)
161 {
162     if (this≠&a)
163     {
164         delete[] array;
165         mallocSize=a.mallocSize;
166         numofItems=a.numofItems;
167         virtualZero=a.virtualZero;
168         array=new T[mallocSize];
169         for (unsigned i=0;i<numofItems;i++)
170         {
171             array[i]=a.array[i];
172         }
173     }
174     return *this;
175 };
176
177 //逻辑=重载
178 template <class T>
179 bool DynamicVector<T>::operator == (const DynamicVector<T>& dv) const
180 {
181     //先比较起始的下标是否相等
182     if(virtualZero≠dv.virtualZero)
183     {
184         return false;
185     }
186     else if(virtualZero=dv.virtualZero)
187     {
188         //再比较已存的元素是否对应相等
189         for(unsigned i=0;i<numofItems;i++)
190         {
191             if(array[i]≠dv.array[i])
192             {
193                 return false;
194             }
195         }
196         return true;
197     }
198 };

```

```

199
200 //StudybarCommentBegin
201 int main()
202 {
203
204     DynamicVector<int> ra(-2);
205     int i,n;
206     cin>>n;
207     ra.push_back(-3);
208     ra.push_back(-2);
209     ra.push_back(-1);
210     for ( i = 0; i < n; i++)
211     {
212         ra.push_back(i);
213     }
214     cout<<"\n malloSize is "<<ra.capacity();
215     cout<<"\n numofItems is "<<ra.length();
216     cout<<"\n StartIndex is " << ra.firstIndex()<<endl;
217     for ( i = -2; i < n+1; i++)
218     {
219         cout<<ra[i]<<" ";
220     }
221     cout<<endl;
222     DynamicVector<int> raCopy(ra);
223     cout<<"\n malloSize is "<<raCopy.capacity();
224     cout<<"\n numofItems is "<<raCopy.length();
225     cout<<"\n StartIndex is " << raCopy.firstIndex()<<endl;
226     cout<<endl;
227     for ( i = -2; i < n+1; i++)
228     {   cout<<+ra[i]<<" ";
229     }
230     cout<<endl;
231     for ( i = -2; i < n+1; i++)
232     {   cout<<raCopy[i]<<" ";
233     }
234
235     raCopy=ra;
236     if (ra==raCopy)   cout<<"\n ra = raCopy";
237     else cout<<"\n ra ≠ raCopy";
238
239
240     ra[-2]=100;
241
242     if (ra==raCopy)   cout<<"\n ra = raCopy";
243     else cout<<"\n ra ≠ raCopy";
244
245     raCopy.push_back(ra);
246     cout<<endl;
247     int firstI=raCopy.firstIndex();
248     for ( i = 0; i < raCopy.length() ; i++)
249     {   cout<<raCopy[i+firstI ]<<" ";
250     }
251     return 0;
252 }
253 //StudybarCommentEnd
254
255

```